L01, Group 06
Melody Yang 400307007
Cynthia Sun 400238765

# MECHTRON 3TB4

## Lab Tutorial 4 Report
## Lab 4 Prelab Report

Date: March 20, 2026

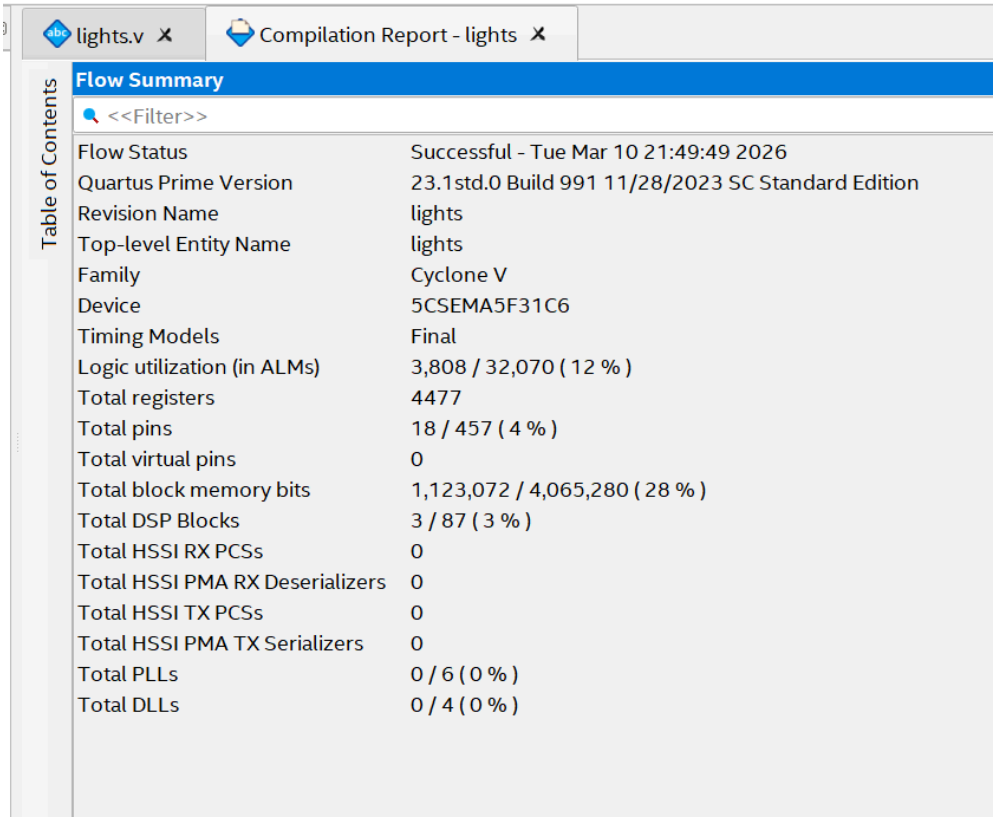
# Lab Tutorial 4 Report

## Overview

The goal of this lab tutorial is to design a System-on-Programmable-Chip (SOPC) on the DE1-SoC FPGA board. A software program written in C is run on a Nios V processor, and the hardware signals will be analyzed during simulation. The lab tutorial is divided into three main sections, which include using the Platform Designer, Ashling RiscFree IDE, and SignalTap Logic Analyzer. The Platform Designer section focuses on developing a system within the FPGA that will be used to implement a hardware system. The Ashling RiscFree IDE section focuses on coding a program in embedded C and running it on a Nios V processor. The SignalTap Logic Analyzer section focuses on debugging and monitoring hardware signals while a program is running.

## Part 1: Platform Designer System Integration Tool

In this section, the hardware system for the project was designed using the Platform Designer tool within Quartus Prime. A Nios V Embedded Processor system was designed by adding components, including the Nios V processor, on-chip memory, two parallel I/O interfaces, and a JTAG UART interface. All the components were interconnected using the Avalon interconnect fabric. The required clock and reset signals were also configured appropriately. The switches and LEDs on the DE1-SoC board were interfaced to the input and output PIO interfaces to enable interaction between the processor and the external hardware components. The base addresses were automatically assigned to the components, and the reset vector was configured to use the on-chip memory. The system was then generated to produce the files required for synthesis. The designed module was then instantiated in a Verilog file and compiled using Quartus Prime before it was downloaded to the FPGA board.



lights.v | Compilation Report - lights

Table of Contents

**Flow Summary**

<<Filter>>

| | |
|---|---|
| Flow Status | Successful - Tue Mar 10 21:49:49 2026 |
| Quartus Prime Version | 23.1std.0 Build 991 11/28/2023 SC Standard Edition |
| Revision Name | lights |
| Top-level Entity Name | lights |
| Family | Cyclone V |
| Device | 5CSEMA5F31C6 |
| Timing Models | Final |
| Logic utilization (in ALMs) | 3,808 / 32,070 ( 12 % ) |
| Total registers | 4477 |
| Total pins | 18 / 457 ( 4 % ) |
| Total virtual pins | 0 |
| Total block memory bits | 1,123,072 / 4,065,280 ( 28 % ) |
| Total DSP Blocks | 3 / 87 ( 3 % ) |
| Total HSSI RX PCSs | 0 |
| Total HSSI PMA RX Deserializers | 0 |
| Total HSSI TX PCSs | 0 |
| Total HSSI PMA TX Serializers | 0 |
| Total PLLs | 0 / 6 ( 0 % ) |
| Total DLLs | 0 / 4 ( 0 % ) |

*Figure 1. Screenshot of the Lights module compilation report.*

*Table 1. Summary for Components for the Lights Module in PreLab 4.*

| **Feature** | **Quantity** |
|---|---|
| Total number of logic elements | 3808 |
| Total number of registers | 4477 |
| Total number of pins | 18 |
| Max number of logic elements that can fit on FPGA | 32,070 |

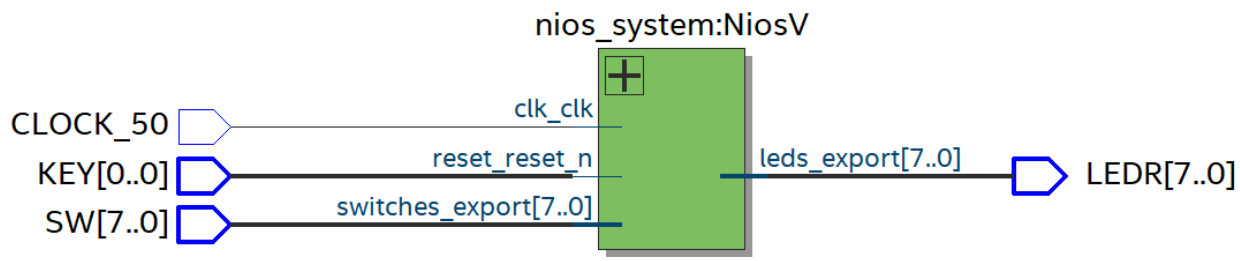



*Figure 2. Screenshot of the implemented Lights Module.*

lights.v  Compilation Report - lights

```
// Implements a simple Nios V system for the DE-series board.
// Inputs: SW7-0 are parallel port inputs to the Nios V system
// CLOCK_50 is the system clock
// KEY0 is the active-low system reset
// Outputs: LEDR7-0 are parallel port outputs from the Nios V system
module lights (CLOCK_50, SW, KEY, LEDR);
   input CLOCK_50;
   input [7:0] SW;
   input [0:0] KEY;
   output [7:0] LEDR;
// Instantiate the Nios V system module generated by the Platform Designer tool:
   nios_system NiosV (
   .clk_clk(CLOCK_50),
   .reset_reset_n(KEY),
   .switches_export(SW),
   .leds_export(LEDR));
endmodule
```

*Figure 3. Screenshot of the Lights Module.*

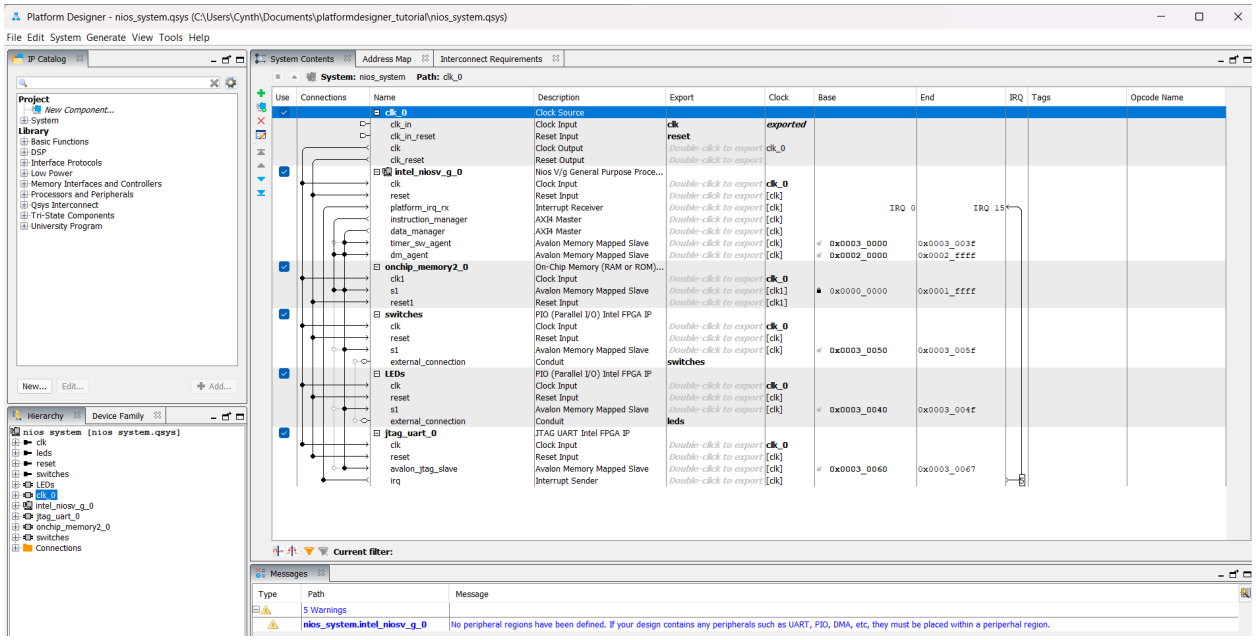



*Figure 4. Screenshot of the Component Connections, IRQ and Export.*

# Part 2: Ashling RiscFree IDE

The second part of the lab tutorial involves writing software to run on top of the Nios V processor. First, a Board Support Package (BSP) was created using the Nios V BSP editor based on the hardware configuration generated by the Platform Designer tool. A software development package is required to develop software for the FPGA using the Nios V processor. After creating the BSP package, an app project was created and imported into the Ashling RiscFree IDE software development package. An embedded software program was written to drive the LEDs by sending a value to the LED peripheral's base address in memory. The software was compiled and run on the DE1-SoC board using a JTAG debugging terminal to view the output of the software.

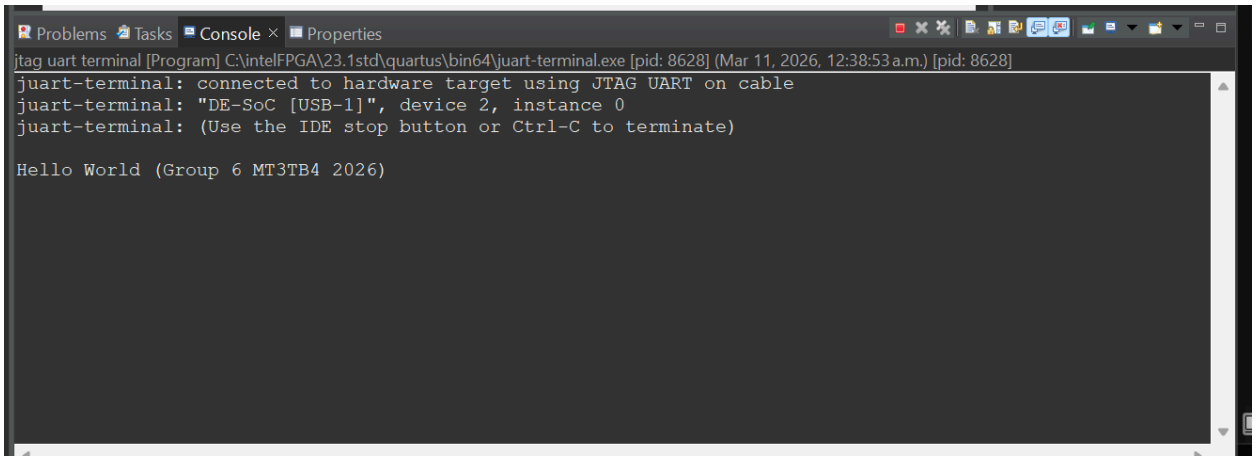



*Figure 5. Screenshot of the Nios Console output.*

```c
#include "system.h"
#include <unistd.h>
#include <stdio.h>

int main()
{
    char * LEDs = (char *) LEDS_BASE;
    volatile char * SWs = (char *) SWITCHES_BASE;

    printf("Hello World (Group 6 MT3TB4 2026) \n");
    printf("Starting LED blink program...\n");

    while (1) {
        //*LEDs = 0xFF; // turn all LEDs on
        //usleep(1000 * 1000);
        //*LEDs = 0x00; // turn all LEDs off
        //usleep(1000 * 1000);
        *LEDs = *SWs;
    }
}
```

*Figure 6. Screenshot of the module where the switches are used to control the LEDs.*

# Part 3: Signal Tap Logic Analyzer

The third part of the lab tutorial involves using the SignalTap Logic Analyzer to observe and debug the internal signals within the FPGA while the system is operational. A configuration file for the SignalTap was designed using Quartus Prime and was connected to the system clock signal. Internal signals such as the chip select signal, address signal, writedata signal, data_out signal, write_n signal, and reset_n signal associated with the LED peripheral were added as nodes for observation within the system. A trigger condition was added for the signal tap logic analyzer such that the analysis would take place while the LED peripheral was accessed within the system. The configuration file was then saved, and the project recompiled and loaded back into the FPGA board. When the LED blinking program was executed on the FPGA board, the signal tap logic analyzer observed the signal and the manner in which the processor interacted with the LED hardware within the system.

## Changing Values of data_out and writedata

The signal tap results observe how the values of writedata and data_out changed for different switch input combinations. Toggling the switches changes the input data presented to the system, and the corresponding bus values can be verified in Signal Tap. The figures below are used to confirm that the correct switch states are captured in the design.

When all switches are off, as shown in the figure below, the switch input corresponds to 0h = 0000000 with the resulting writedata and data_out states reflected as 0.

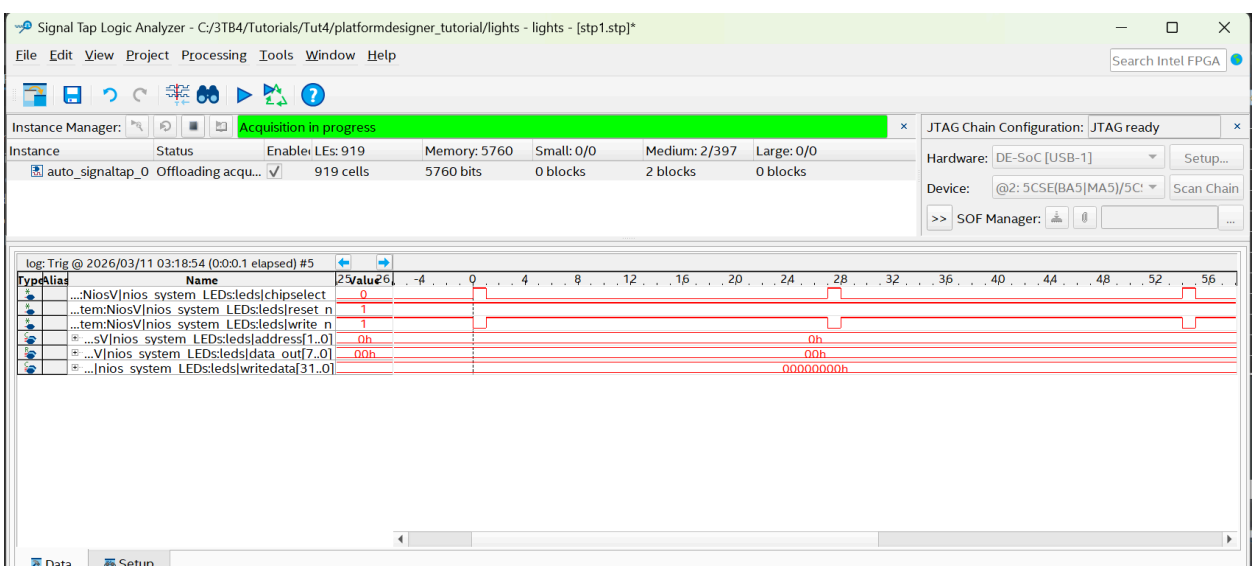



*Figure 7. Signal Tap result with all switches off.*

When all the switches are on, the switch input becomes FFh = 11111111, which causes write data to show a maximum 8-bit switch value and data_out to become similar, as shown with 000000FFh.

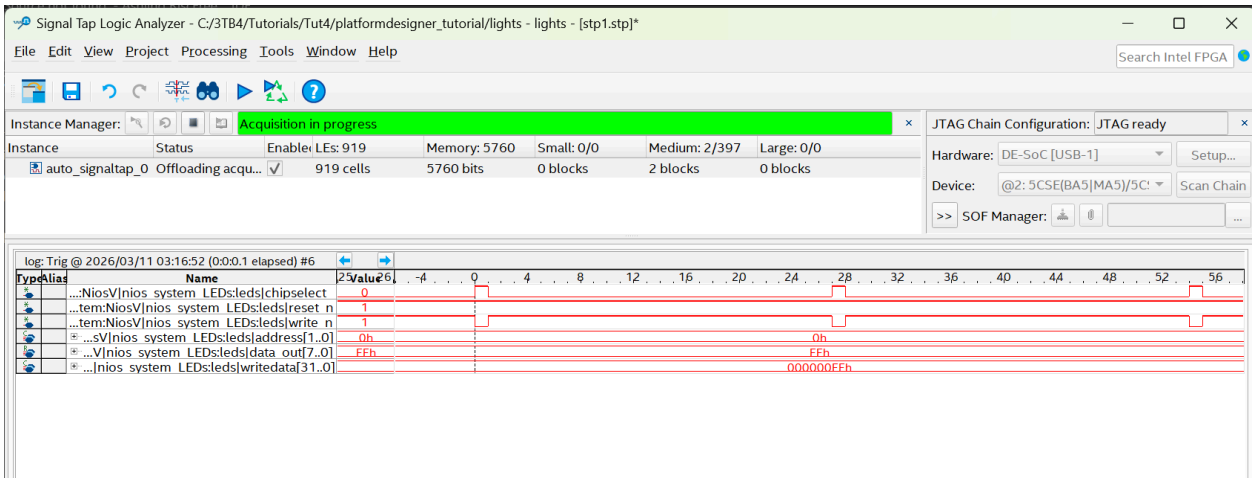



*Figure 8. Signal Tap result with all switches on.*

When the first four switches are on with 0Fh = 00001111, the signal tap captures writedata corresponding to a lower-nibble active pattern while data_out matches it correspondingly with 0000000Fh.

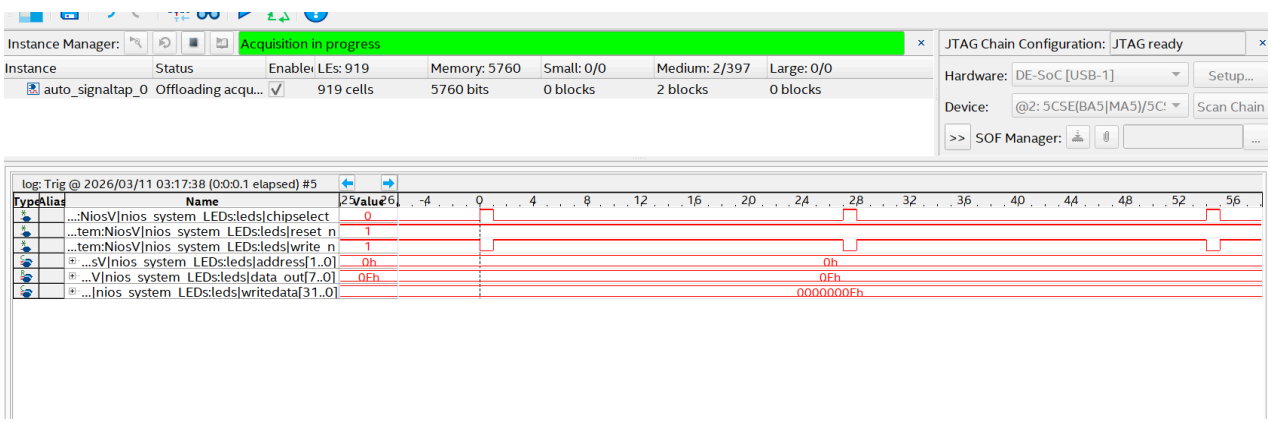



*Figure 9. Signal Tap result with first four switches on (SW[3:0] on).*

When the last four switches are on with F0h = 11110000, the signal tap captures writedata corresponding to an upper-nibble active pattern while data_out matches it correspondingly with 000000F0h.

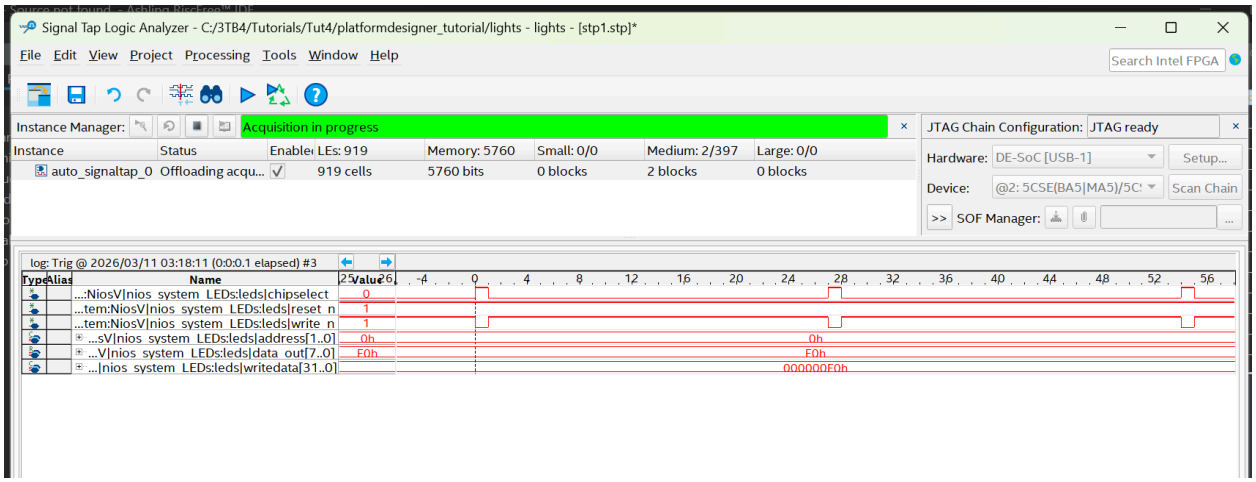



*Figure 10. Signal Tap result with last four switches on (SW[7:4] on).*

The following tap results with only even switches on (SW[0], SW[2], SW[4], SW[6] on), which results in an alternating even pattern of 1 and 0 of 55h = 01010101. The corresponding writedata value becomes an alternating pattern that corresponds to the data_out.

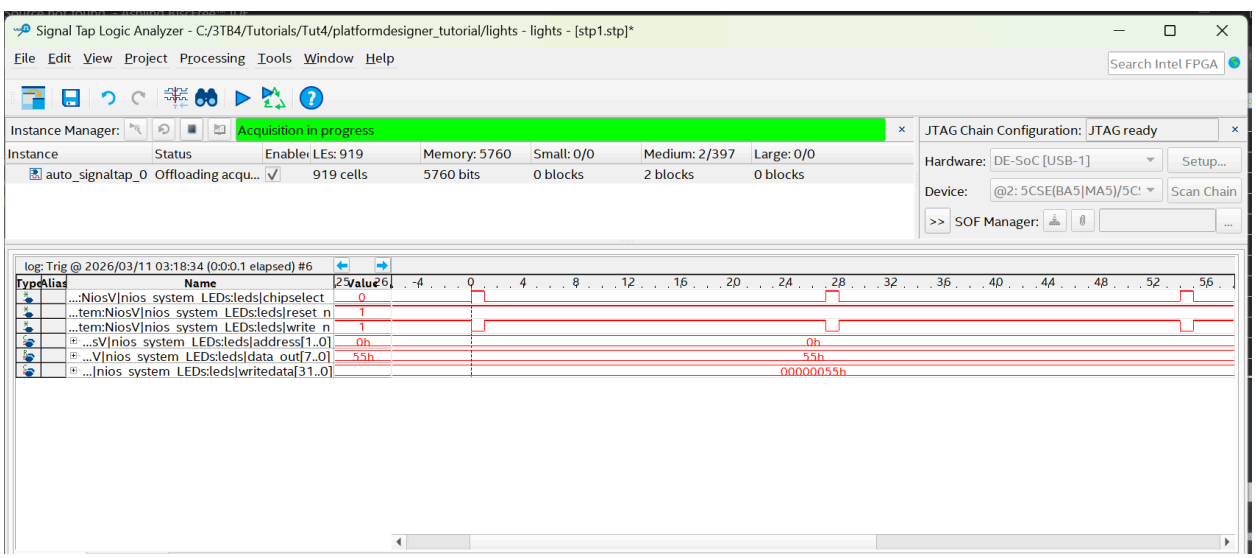



*Figure 11. Signal Tap results with only even switches on (SW[0], SW[2], SW[4], SW[6] on).*

# Questions

1. Which of the devices in this system were master devices and slave devices?
   The master device is used to control the bus, whereas the slave responds to the requests and gives or stores data. In this case, the NIOSV processor is the master device that sends signals to initiate read or write operations on the Avalon bus, and the SDRAM Controller responds to its requests. This also includes the other memory peripherals.

2. Which of the following components used in this tutorial were software and which were hardware? If you think a part belongs in both, explain why.
   a. Quartus
      Quartus is used to compile Verilog code and VHDL designs, synthesizing hardware circuits and programming the FPGA. However, this is primarily used as a hardware tool but also interfaces with software tools such as Signal Tap analyzer and NIOSV, indirectly supporting software development.

   b. Platform Designer
      Platform designer links the system architecture, connecting CPU, SDRAM, peripherals, and so forth, defining the Avalon bus connections, which is mostly hardware-based but lay the groundwork for software connections. Furthermore, it is used to generate system.h files through the sopc_system.sopcinfo file used in the BSP editor, which is used in the software aspect of the hello_world.c file as an included header file. Therefore, platform designers are both software and hardware-based.

   c. Ashling RiscFree IDE
      RiscFree IDE is the software application used for the NIOSV system that can be imported onto the NIOS V hardware aspect of the system. It is used to write C programs and compile or debug the code, which can then be uploaded to the NIOSV processor. This application is software and interacts with the hardware, but doesn't design the hardware.

   d. Signal Tap Logic Analyzer
      The Signal Tap Logic Analyzer has both hardware and software aspects. It's synthesized into the FPGA design, adding nodes/probes that monitor the internal signals in real-time. However, it utilizes the software application configured into QUARTUS to select signals or view waveforms. Therefore, it's a hardware debugging tool that monitors the FPGA signals.

# Lab 4 Prelab Report

## Overview

This prelab focuses on understanding the SDRAM addressing, system architecture, and preparing the hardware and software components needed for Lab 4. The capacity and addressing of the SDRAM device on the DE1-SoC board were analyzed. Since the SDRAM is used to store 64 MB of memory, calculations are made to determine the highest possible address and how byte addresses from the processor are converted into word addresses required by the SDRAM device. Then, the wrapper module (SDRAM_Controller.v) was created, connecting the SDRAM controller to the Avalon interface. To better understand how the processor, controller, and memory interact, a schematic of the circuit of the system was created. Additionally, a software program in C was written to test the SDRAM memory, which writes and reads different data types, such as char, short, and int, to verify that the data stored in the SDRAM can be received properly.

## Questions

1. Considering that the SDRAM chip you will be using has a capacity of 64M bytes, and assuming that the lowest address of the SDRAM chip is 0x00000000, what will be the highest byte address?

   <u>Step 1:</u> Calculate the total number of bytes in 64 MB

   $$64\,MB = 64 \times 2^{20} = 2^{26}\,bytes = 67,108,864\,bytes$$

   <u>Step 2:</u> Since the lowest address is 0x00000000, the highest address can be calculated using the following formula:

   $$Highest\ Address = 2^{26} - 1 = 67,108,864 - 1 = 67,108,863$$

   <u>Step 3:</u> Convert $67,108,864$ to hexadecimal:

   $$67,108,864 = 0x03FFFFFF$$

   Therefore, the highest byte address is **0x03FFFFFF**.

2. Considering that the chip accepts word addresses, what addresses should the chip see on its input when the processor requests data from the lowest and the highest byte in the memory? (Assume that the base address is 0x00000000 for this question. The SOPC

will assign a different base address as part of the design process, as discussed later in the lab.)

Using the conversion factor where one word is equal to two bytes, the lowest and highest byte can be calculated using division or bit-shifting to the right by 1:

$$Lowest\ Byte\ = \frac{0x00000000}{2}\ = 0x00000000$$

$$Highest\ Byte\ = \frac{0x03FFFFFF}{2}\ = 0x01FFFFFF$$

3. When you write a C program to access memory, the compiler generates byte addresses (i.e. address 0x0000 will be the first byte and address 0x0001 will be the second byte). However, since the SDRAM that is used in the lab accepts a word address, the Avalon interconnect will automatically convert the byte address to a word address. What address will appear on the SDRAM address lines when the second byte is accessed?

   Converting the second byte address to word address:

   $$Second\ Byte\ = \frac{0x0001}{2}\ = 0x0000$$ (rounded down)

This indicates that the first address byte (0x0000) and the second address byte (0x0001) will appear as the same word address. Avalon's byte_enable signal will indicate which byte within the word is being accessed.

# Part 1: SDRAM Controller

The SDRAM Controller is in the module named SDRAM_Controller.v, which is an interface between the Nios V system and the SDRAM chip connected to the DE1-SoC board. It wraps the provided SDRAM controller and specifies the input/output ports to integrate the Avalon memory-mapped interface with the hardware side's SDRAM signals. On the Avalon side, the controller processes signals, such as the address, read/write signals, byte enable signals, data signals, etc., from the processor side. On the hardware side, the module processes the SDRAM signals such as the clock, chip enable, row address strobe, column address strobe, write enable, bank addresses, data signals, etc., to execute the appropriate command sequences to the SDRAM to execute the memory operations. The controller on the hardware side processes the SDRAM signals, such as the clock, chip enable, row address strobe, column address strobe, write enable, bank addresses, data signals, etc., to execute the appropriate command sequences to the SDRAM to execute the memory operations.

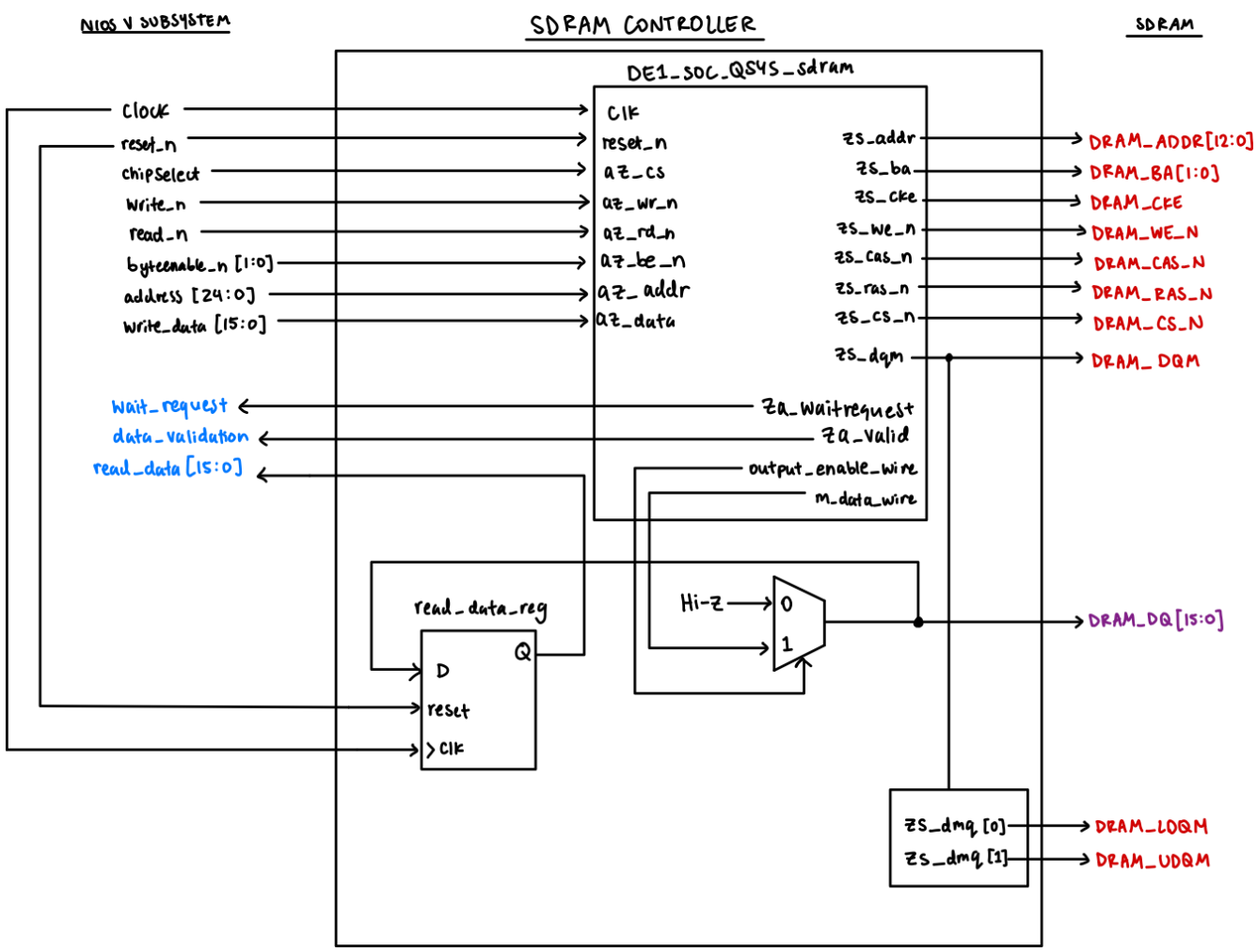



*Figure 12. Drawn schematic of Circuit for SDRAM_Controller.v.*

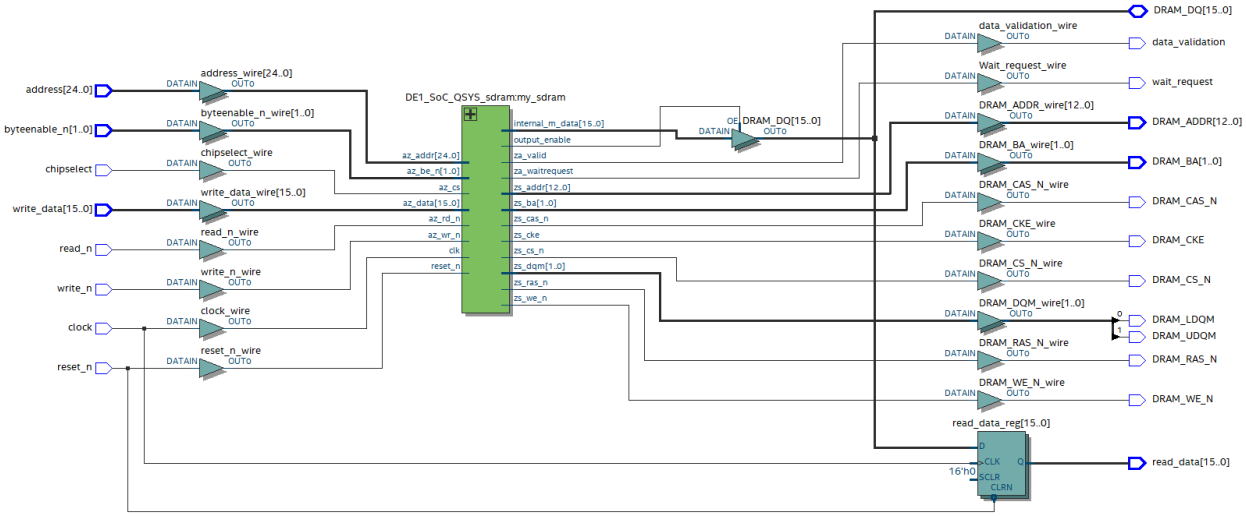



*Figure 13. Schematic of Circuit for SDRAM_Controller.v.*

**Flow Summary**

<<Filter>>

| | |
|---|---|
| Flow Status | Successful - Fri Mar 20 18:05:19 2026 |
| Quartus Prime Version | 23.1std.0 Build 991 11/28/2023 SC Standard Edition |
| Revision Name | lab4 |
| Top-level Entity Name | SDRAM_Controller |
| Family | Cyclone V |
| Device | 5CSEMA5F31C6 |
| Timing Models | Final |
| Logic utilization (in ALMs) | 184 / 32,070 ( < 1 % ) |
| Total registers | 300 |
| Total pins | 104 / 457 ( 23 % ) |
| Total virtual pins | 0 |
| Total block memory bits | 0 / 4,065,280 ( 0 % ) |
| Total DSP Blocks | 0 / 87 ( 0 % ) |
| Total HSSI RX PCSs | 0 |
| Total HSSI PMA RX Deserializers | 0 |
| Total HSSI TX PCSs | 0 |
| Total HSSI PMA TX Serializers | 0 |
| Total PLLs | 0 / 6 ( 0 % ) |
| Total DLLs | 0 / 4 ( 0 % ) |

*Figure 14. Screenshot of the SDRAM_Controller.v module compilation report.*

# Part 2: C Program for the Nios V System

The C program used in the Nios V system is designed to test the communication between the processor and SDRAM memory using the SDRAM controller. The program is created and compiled using the Ashling RiscFree. In the program, the Nios V processor writes data to specific SDRAM addresses and then reads back that data to ensure that memory read/write operations are functioning correctly. In addition, different data types, such as char, short, and int, are used to read and write memory using different data sizes, which represent 8-bit, 16-bit, and 32-bit data types. These data types are used to ensure that the SDRAM controller is able to support different data sizes and that the processor is able to read and write data to SDRAM memory correctly. The testing is performed to ensure that the SDRAM controller is able to support different data sizes and that the processor is able to read and write data to SDRAM memory correctly.

hello_world.c

```c
#include "system.h"
#include <stdio.h>

#define BASE SDRAM_CONTROLLER_0_BASE
#define SDRAM_MAXNUM_WORDS SDRAM_CONTROLLER_0_SPAN/2
    //the size of the SDRAM on DE1-SoC is 64MB =67108864 Byte (SDRAM_SPAN)

#define Testing_MAXNUM_WORDS SDRAM_CONTROLLER_0_SPAN/2 //5

//To test part of the memory, for example, to test the first 1/10 of the memory:
//#define Testing_MAXNUM_WORDS   SDRAM_CONTROLLER_0_SPAN/20


int main() {
    printf("Hello from MT3TB4 Group 6!\n\n\n");
    printf("Testing %d  words out of Maximum %d words On SDRAM......\n\n\n", Testing_MAXNUM_WORDS, SDRAM_MAXNUM_WORDS);

    int i;
    int char_err_num = 0, short_err_num = 0, int_err_num = 0;

    char * char_ptr;
    char aChar;
    short *short_ptr;
    short aShort;
    int *int_ptr;
    int aInt;

    int charsize, shortsize, intsize;
    charsize = sizeof(aChar);
    shortsize = sizeof(aShort);
    intsize = sizeof(aInt);

    printf("the sizeof char, short, int in bytes are: %d, %d, %d\n", charsize, shortsize, intsize);


    //----------------TEST CHAR-----------------------------------
    while(1){
    printf("\n writing chars.....\n");
```

*Figure 15. Screenshot of the hello_world.c Module.*

## Part 3: Top-Level Module

The top-level module combines all the elements of the FPGA system with the physical hardware of the DE1-SoC board. In the top-level module of the Lab 4 system, the Nios V system, which is created by the Platform Designer, is instantiated and connected to the board's clock, reset signals, input/output devices, and so on. The SDRAM controller component is also connected to the pins of the FPGA board to enable the processor to access the external memory. Other board peripherals, like switches and LEDs, may also be connected to the system for the purpose of debugging the system. The top-level design is the main module of the system, which combines the processor system, the custom-designed SDRAM controller, and the hardware peripherals of the FPGA board before the system is compiled and programmed.

**Flow Summary**

<<Filter>>

| | |
|---|---|
| Flow Status | Successful - Fri Mar 20 19:55:04 2026 |
| Quartus Prime Version | 23.1std.0 Build 991 11/28/2023 SC Standard Edition |
| Revision Name | lab4 |
| Top-level Entity Name | lab4 |
| Family | Cyclone V |
| Device | 5CSEMA5F31C6 |
| Timing Models | Final |
| Logic utilization (in ALMs) | 4,129 / 32,070 ( 13 % ) |
| Total registers | 4851 |
| Total pins | 57 / 457 ( 12 % ) |
| Total virtual pins | 0 |
| Total block memory bits | 1,450,752 / 4,065,280 ( 36 % ) |
| Total DSP Blocks | 3 / 87 ( 3 % ) |
| Total HSSI RX PCSs | 0 |
| Total HSSI PMA RX Deserializers | 0 |
| Total HSSI TX PCSs | 0 |
| Total HSSI PMA TX Serializers | 0 |
| Total PLLs | 1 / 6 ( 17 % ) |
| Total DLLs | 0 / 4 ( 0 % ) |

*Figure 16. Screenshot of the Top-Level module compilation report.*

*Table 2. Summary for Components for the Top-Level Module in PreLab 4.*

| Feature | Quantity |
|---|---|
| Total number of logic elements | 4129 |
| Total number of registers | 4851 |
| Total number of pins | 57 |
| Max number of logic elements that can fit on FPGA | 32,070 |

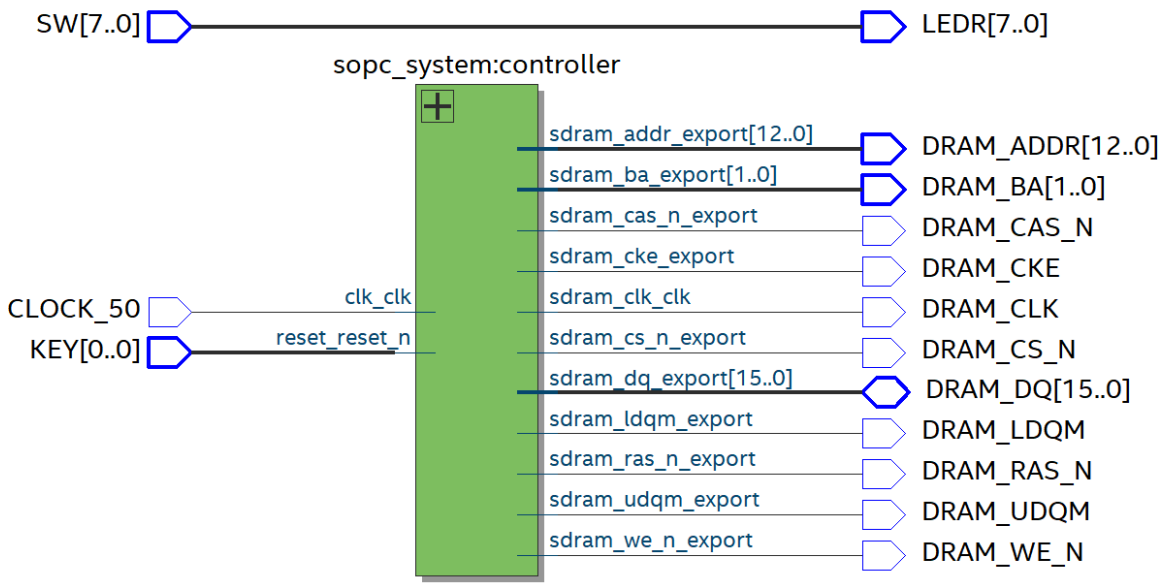



*Figure 17. Screenshot of the implemented Top-Level Module.*

```
module lab4 (
input          CLOCK_50,
input    [0:0] KEY,
input    [7:0] SW,               //for reset
output   [7:0] LEDR,

// Bidirectionals
inout    [15:0]      DRAM_DQ,

// Outputs

output      [12:0]   DRAM_ADDR,
output      [1:0]    DRAM_BA,
output               DRAM_LDQM,  //data mask; when it is low, the DQ is valid for reading and writing.
output               DRAM_UDQM,
output               DRAM_RAS_N,
output               DRAM_CAS_N,
output               DRAM_CLK,
output               DRAM_CKE,
output               DRAM_WE_N,
output               DRAM_CS_N

);


// Internal Wires


assign LEDR=SW;

//Instantiate your sopc_system module generated by Platform Designer.

```

*Figure 18. Screenshot of the module of the Top-Level Module.*

# Code

## Lab Tutorial 4

### Platform Designer System Integration Tool

```
// Implements a simple Nios V system for the DE-series board.
// Inputs: SW7-0 are parallel port inputs to the Nios V system
// CLOCK_50 is the system clock
// KEY0 is the active-low system reset
// Outputs: LEDR7-0 are parallel port outputs from the Nios V system
module lights (CLOCK_50, SW, KEY, LEDR);
	input CLOCK_50;
	input [7:0] SW;
	input [0:0] KEY;
	output [7:0] LEDR;
	// Instantiate the Nios V system module generated by the Platform Designer tool:
	nios_system NiosV (
	.clk_clk(CLOCK_50),
	.reset_reset_n(KEY),
	.switches_export(SW),
	.leds_export(LEDR));
endmodule
```

## Blinking Lights (Ashling RiscFree IDE)

```
#include "system.h"
#include <unistd.h>
#include <stdio.h>

int main()
{
    char * LEDs = (char *) LEDS_BASE;
    volatile char * SWs = (char *) SWITCHES_BASE;

    printf("Hello World (Group 6 MT3TB4 2026) \n");

    while (1) {
        *LEDs = 0xFF; // turn all LEDs on
        usleep(1000 * 1000);
        *LEDs = 0x00; // turn all LEDs off
        usleep(1000 * 1000);
        //*LEDs = *SWs;
    }
}
```

## Switches Control LEDs (Ashling RiscFree IDE)

```
#include "system.h"
#include <unistd.h>
#include <stdio.h>

int main()
{
    char * LEDs = (char *) LEDS_BASE;
    volatile char * SWs = (char *) SWITCHES_BASE;

    printf("Hello World (Group 6 MT3TB4 2026) \n");

    while (1) {
        *LEDs = *SWs;
    }
```

# Prelab 4

## SDRAM Controller

```
/*****************************************************************************
 *                                                                           *
 * Module:      SDRAM_Controller                                             *
 * Description:                                                              *
 *      This module is used for the sram controller for MT3TB4 Lab 4         *
 *                                                                           *
 *****************************************************************************/
`timescale 1ns / 1ps

module SDRAM_Controller (  //the ports sequence follows the sequence of wires in the diagram
in the lab manual
                //signals through Avalon Interface:
                input                   clock,
                input                   reset_n,
                input                   chipselect,
                input                           write_n,
                input                           read_n,
                input     [1:0]   byteenable_n,
                input     [24:0]  address,   //2's power of 25 is  33,554,432 that is 32M.  (16 bits
word, that makes capable for 64MBytes)
                input     [15:0]  write_data,
                output [15:0]     read_data,
                output                  wait_request,
                output                  data_validation,

                //signals between Conroller and SDRAM:

                inout     [15:0]  DRAM_DQ,
                output [12:0]     DRAM_ADDR,
                output [1:0]      DRAM_BA,
        //      output                  DRAM_CLK,
                output                  DRAM_CKE,
                output                  DRAM_LDQM,
                output                  DRAM_UDQM,
                output                  DRAM_WE_N,
                output                  DRAM_CAS_N,
                output                  DRAM_RAS_N,
                output                  DRAM_CS_N
);
```

```
wire                    clock_wire/*synthesis keep*/;
wire                    reset_n_wire/*synthesis keep*/;
wire                    chipselect_wire/*synthesis keep*/;
wire                    write_n_wire/*synthesis keep*/;
wire                    read_n_wire/*synthesis keep*/;
wire    [1:0]       byteenable_n_wire/*synthesis keep*/;
wire    [24:0]  address_wire/*synthesis keep*/;   //2's power of 25 is  33,554,432
that is 32M.  (16 bits word, that makes capable for 64MBytes)
wire    [15:0]  write_data_wire/*synthesis keep*/;
wire                    Wait_request_wire/*synthesis keep*/;
wire                    data_validation_wire/*synthesis keep*/;

reg     [15:0]  read_data_reg/*synthesis preserve*/;
//wire  [15:0]  read_data_wire/*synthesis keep*/;

//signals between Conroller and SDRAM:

wire    [15:0]  DRAM_DQ_wire/*synthesis keep*/;
wire    [12:0]  DRAM_ADDR_wire/*synthesis keep*/;
wire    [1:0]       DRAM_BA_wire/*synthesis keep*/;

wire                    DRAM_CKE_wire/*synthesis keep*/;

wire    [1:0]   DRAM_DQM_wire/*synthesis keep*/;        //higher bit for UDQM
, lower bit for LDQM
wire                    DRAM_WE_N_wire/*synthesis keep*/;
wire                    DRAM_CAS_N_wire/*synthesis keep*/;
wire                    DRAM_RAS_N_wire/*synthesis keep*/;
wire                    DRAM_CS_N_wire/*synthesis keep*/;

wire [15:0]         m_data_wire;
wire                    output_enable_wire;

DE1_SoC_QSYS_sdram  my_sdram (
   // inputs:
    .az_addr(address_wire),
    .az_be_n(byteenable_n_wire),
    .az_cs(chipselect_wire),
    .az_data(write_data_wire),
    .az_rd_n(read_n_wire),
    .az_wr_n(write_n_wire),
    .clk(clock_wire),
    .reset_n(reset_n_wire),
```

```
                    // outputs:
                    //.za_data(read_data_wire),    //can not get read_data from here.
                     .za_valid(data_validation_wire),
                     .za_waitrequest(Wait_request_wire),
                     .zs_addr(DRAM_ADDR_wire),
                     .zs_ba(DRAM_BA_wire),
                     .zs_cas_n(DRAM_CAS_N_wire),
                     .zs_cke(DRAM_CKE_wire),
                     .zs_cs_n(DRAM_CS_N_wire),
                     //.zs_dq(DRAM_DQ_wire),
                     .zs_dqm(DRAM_DQM_wire),
                     .zs_ras_n(DRAM_RAS_N_wire),
                     .zs_we_n(DRAM_WE_N_wire),

.output_enable(output_enable_wire),

.internal_m_data(m_data_wire)
                    );

        //=============Make connections =====================

                assign                  DRAM_DQ=output_enable_wire?m_data_wire:{16{1'bz}} ;

                //this way works  , get data from za_data port does not works.
                always @(posedge clock or negedge reset_n)   //must be synchronize with the
clock!
                 begin
                        if (reset_n == 0)
                                read_data_reg<= 0;
                        else
                                read_data_reg <= DRAM_DQ;
                 end

                assign                  clock_wire=clock;
                assign                  reset_n_wire=reset_n;
                assign                  chipselect_wire=chipselect;
                assign                  write_n_wire=write_n;
                assign                  read_n_wire=read_n;
                assign                  byteenable_n_wire=byteenable_n;
                assign                  address_wire=address;   //2's power of 25 is  33,554,432
that is 32M.  (16 bits word, that makes capable for 64MBytes)
                assign                  write_data_wire=write_data;

                //output signals
```

```
assign		read_data=read_data_reg;
//assign		read_data=read_data_wire;	//can not get read_data from the za_data port

assign		wait_request=Wait_request_wire;
assign		data_validation=data_validation_wire;

//signals between Conroller and SDRAM:

assign		DRAM_UDQM=DRAM_DQM_wire[1];		//higher bit for UDQM , lower bit for LDQM
assign		DRAM_LDQM=DRAM_DQM_wire[0];

//  ********** More  necessary connections that YOUR NEED TO MAKE for Lab 4 project **************

assign		DRAM_ADDR=DRAM_ADDR_wire;
assign		DRAM_BA	=DRAM_BA_wire;
assign		DRAM_CKE	=DRAM_CKE_wire;
assign		DRAM_WE_N=DRAM_WE_N_wire;
assign		DRAM_CAS_N=DRAM_CAS_N_wire;
assign		DRAM_RAS_N=DRAM_RAS_N_wire;
assign		DRAM_CS_N=DRAM_CS_N_wire;



endmodule
```

## C Program for the Nios V System

```
#include "system.h"
#include <stdio.h>
#define BASE SDRAM_CONTROLLER_0_BASE
#define SDRAM_MAXNUM_WORDS SDRAM_CONTROLLER_0_SPAN/2
        //the size of the SDRAM on DE1-SoC is 64MB =67108864 Byte (SDRAM_SPAN)
#define Testing_MAXNUM_WORDS SDRAM_CONTROLLER_0_SPAN/2 //5
//To test part of the memory, for example, to test the first 1/10 of the memory:
//#define Testing_MAXNUM_WORDS   SDRAM_CONTROLLER_0_SPAN/20
int main() {
        printf("Hello from MT3TB4 Group 6!\n\n\n");
        printf("Testing %d  words out of Maximum %d words On SDRAM......\n\n\n",
Testing_MAXNUM_WORDS, SDRAM_MAXNUM_WORDS);
        int i;
        int char_err_num = 0, short_err_num = 0, int_err_num = 0;
        char * char_ptr;
        char aChar;
        short *short_ptr;
        short aShort;
        int *int_ptr;
        int aInt;
        int charsize, shortsize, intsize;
        charsize = sizeof(aChar);
        shortsize = sizeof(aShort);
        intsize = sizeof(aInt);
        printf("the sizeof char, short, int in bytes are: %d, %d, %d\n", charsize, shortsize, intsize);
        //----------------TEST CHAR----------------------------------
        while(1){
        printf("\n writing chars.....\n");
        for (i = 1; i <= Testing_MAXNUM_WORDS * 2; i++) {
                *(char*) (BASE + i) = i % 128; // to be safe, use 128 rather than 256
        }
        printf("\n testing chars.....\n");
        for (i = 1; i <= Testing_MAXNUM_WORDS * 2; i++) {
                if (*(char*) (BASE + i) != i % 128) { // or .....(char)i,   if not i%128
                        char_err_num++;
                }
        }
        printf("\nTesting  Char: the total numbers of error is : %i\n", char_err_num);
        //----------------TEST SHORT----------------------------------
        printf(" \n writing short......\n");
        for (i = 1; i <= Testing_MAXNUM_WORDS; i++) {
                *(short*) (BASE + i * 2) = i % 32767; // short, uses two bytes
        }
        printf(" \n testing short......\n");
        for (i = 1; i <= Testing_MAXNUM_WORDS; i++) {
                if (*(short*) (BASE + i*2) != i % 32767) { // or .....(short)i,   if not i%32767
                        short_err_num++;
                }
        }
```

```
	printf("\nTesting Short: the total numbers of error is : %i\n",
		short_err_num);
	//---------------TEST INT    ----------------------------------
	printf(" \n writing integer......\n");
	for (i = 1; i <= Testing_MAXNUM_WORDS / 2; i++) {
		*(int*) (BASE + i * 4) = i; // int, use 4 bytes
	}
	printf(" \n testing integer......\n");
	for (i = 1; i <= Testing_MAXNUM_WORDS / 2; i++) {
		if (*(int*) (BASE + i*4) != i) { // or .....(int)i,   if not i
			int_err_num++;
		}
	}
	printf("\nTesting int: the total numbers of error is : %i\n", int_err_num);
	printf("\nMemory test complete. # Char errors: %i. # Short errors: %i. # int errors: %i.",
char_err_num, short_err_num, int_err_num);
	if(!char_err_num && !short_err_num && !int_err_num == 1){
		printf("\nMemory test passed.");
	}else{
		printf("\nMemory test failed.");
	}
	}
	return 0;
}
```

## Top-Level Module

```
/*****************************************************************************
 *                                                                           *
 * Module:      Lab4                                                         *
 * Description:                                                              *
 *      This module is the top level module of MT3TB4 Lab 4                  *
 *                                                                           *
 *****************************************************************************/

module lab4 (
input                           CLOCK_50,
input           [0:0]     KEY,
input    [7:0] SW,              //for reset
output  [7:0] LEDR,

// Bidirectionals
inout           [15:0]          DRAM_DQ,

// Outputs

output          [12:0]   DRAM_ADDR,
output          [1:0]           DRAM_BA,
output                                  DRAM_LDQM,  //data mask; when it is low, the DQ is valid
for reading and writing.
output                                  DRAM_UDQM,
output                                  DRAM_RAS_N,
output                                  DRAM_CAS_N,
output                                  DRAM_CLK,
output                                  DRAM_CKE,
output                                  DRAM_WE_N,
output                                  DRAM_CS_N

);


// Internal Wires


assign LEDR=SW;

//Instantiate your sopc_system module generated by Platform Designer.


sopc_system  controller (
```

```
		// example ports
		.clk_clk(CLOCK_50),		//		clk.clk
		.reset_reset_n(KEY[0]),		//		reset.reset_n
		.sdram_addr_export(DRAM_ADDR),		// .sdram_addr_export
		.sdram_ba_export(DRAM_BA),		//sdram_ba_export
		.sdram_cas_n_export(DRAM_CAS_N), // sdram_cas_n.export
		.sdram_cke_export(DRAM_CKE),	//	sdram_cke.export
		.sdram_clk_clk(DRAM_CLK),	//	sdram_clk.clk
		.sdram_cs_n_export(DRAM_CS_N),	//	sdram_cs_n.export
		.sdram_dq_export(DRAM_DQ),	//	sdram_dq.export
		.sdram_ldqm_export(DRAM_LDQM),	//	sdram_ldqm.export
		.sdram_ras_n_export(DRAM_RAS_N), // sdram_ras_n.export
		.sdram_udqm_export(DRAM_UDQM),	//	sdram_udqm.export
		.sdram_we_n_export(DRAM_WE_N)	//	sdram_we_n.export

	);



endmodule
```